Netflix Data Science Project:

Introduction

This notebook performs a comprehensive Exploratory Data Analysis (EDA) on a Netflix dataset. The goal is to uncover trends in content creation, understand distribution across countries, and categorize content for better insights.

Steps covered:

1. **Data Cleaning:** Handling missing values and duplicates.
2. **Feature Engineering:** Creating new metrics like `year_added`, `cast_count`, and `rating_category`.
3. **Exploratory Analysis:** Statistical summaries and visual inspections.

In [ ]:

```

```

In [ ]:

```

```

In [ ]:

```

```

In [ ]:

```

```

In [ ]:

```

```

1. Environment Setup and Data Loading

In this section, we import the necessary libraries for data manipulation (`pandas`, `numpy`) and visualization (`seaborn`, `matplotlib`). We also load the raw Netflix dataset.

In [298]:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

from warnings import filterwarnings
filterwarnings("ignore")



df = pd.read_csv("dataset/Netflix.csv")
```

2. Initial Data Inspection

Before cleaning, we take a quick look at the first few rows of the data, its dimensions, and the data types of each column to identify potential issues.

In [299]:

In [299]:

```
df.head()
```

Out[299]:

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	genres
0	s1	TV Show	3%	NaN	João Miguel, Bianca Comparato, Michel Gomes, R...	Brazil	14-Aug-20	2020	TV-MA	4	International TV Shows, TV Dramas, TV Sci-Fi &...
1	s10	Movie	1920	Vikram Bhatt	Rajneesh Duggal, Adah Sharma, Indraneil Sengup...	India	15-Dec-17	2008	TV-MA	143	Horror Movies, International Movies, Thrillers
2	s100	Movie	3 Heroines	Iman Brotoseno	Reza Rahadian, Bunga Citra Lestari, Tara Basro...	Indonesia	5-Jan-19	2016	TV-PG	124	Dramas, International Movies, Sports Movies
3	s1000	Movie	Blue Mountain State: The Rise of Thadland	Lev L. Spiro	Alan Ritchson, Darin Brooks, James Cade, Rob R...	United States	1-Mar-16	2016	R	90	Comedies
4	s1001	TV Show	Blue Planet II	NaN	David Attenborough	United Kingdom	3-Dec-18	2017	TV-G	1	British TV Shows, Docuseries, Science & Nature TV

In [300]:

```
df.shape
```

Out[300]:

```
(7787, 12)
```

In [301]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7787 entries, 0 to 7786
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       7787 non-null   object
 1   type          7787 non-null   object
 2   title         7787 non-null   object
 3   director      5398 non-null   object
 4   cast          7069 non-null   object
 5   country       7280 non-null   object
 6   date_added    7777 non-null   object
 7   release_year  7787 non-null   int64 
 8   rating        7780 non-null   object
 9   duration      7787 non-null   int64 
 10  genres        7787 non-null   object
 11  description   7787 non-null   object
dtypes: int64(2), object(10)
memory usage: 730.2+ KB
```

In [302]:

```
df.describe()
```

Out[302]:

	release_year	duration
count	7787.000000	7787.000000
mean	2013.932580	69.122769
std	8.757395	50.950743
min	1925.000000	1.000000
25%	2013.000000	2.000000
50%	2017.000000	88.000000
75%	2018.000000	106.000000
max	2021.000000	312.000000

In [303]:

```
df.isnull().sum()
```

Out[303]:

```
show_id            0
type               0
title              0
director        2389
cast             718
country          507
date_added        10
release_year       0
rating             7
duration           0
genres             0
description        0
dtype: int64
```

In [304]:

```
df.duplicated().sum()
```

Out[304]:

```
np.int64(0)
```

3. Data Cleaning

To ensure the quality of our analysis, we perform the following:

- **Drop Unnecessary Columns:** Removing `show_id` as it serves only as a unique identifier.
- **Impute Missing Values:** Filling gaps in `director`, `cast`, and `country` with "Unknown", and using the mode for `rating` and `date_added`.

In [305]:

```
df.drop(columns=['show_id'], inplace=True)
```

In [306]:

```
df['director'].fillna('Unknown', inplace=True)
df['cast'].fillna('Unknown', inplace=True)
df['country'].fillna('Unknown', inplace=True)
df['rating'].fillna(df['rating'].mode()[0], inplace=True)
df['date_added'].fillna(df['date_added'].mode()[0], inplace=True)
```

In [ ]:

```

```

4. Feature Engineering

We create new features to extract more value from the existing data:

1. **Date Extraction:** Converting `date_added` to datetime objects to extract year and month.
2. **Complexity Metrics:** Calculating the number of cast members and genres per title.
3. **Categorization:** Simplifying the content ratings into broader categories: **Adult**, **Teen**, and **Kids**.
4. **Localization:** Extracting the primary country of origin from the `country` column.

In [307]:

```
df['date_added'] = pd.to_datetime(df['date_added'])

df['year_added'] = df['date_added'].dt.year
df['month_added'] = df['date_added'].dt.month
```

In [308]:

```
df.drop(columns=['date_added'], inplace=True)
```

In [309]:

```
df['cast_count'] = df['cast'].apply(lambda x: len(x.split(',')))
df['genre_count'] = df['genres'].apply(lambda x: len(x.split(',')))
```

In [310]:

```
df['duration_type'] = df['type']

df.rename(columns={'duration':'duration_value'}, inplace=True)
```

In [311]:

```
df['country_main'] = df['country'].apply(lambda x: x.split(',')[0])
```

In [312]:

```
def simplify_rating(r):
    if r in ['TV-MA','R','NC-17']:
        return "Adult"
    elif r in ['PG-13','TV-14']:
        return "Teen"
    else:
        return "Kids"

df['rating_category'] = df['rating'].apply(simplify_rating)
```

In [313]:

```
df.drop(columns=['rating'], inplace=True)
```

5. Exploratory Data Analysis & Visualization

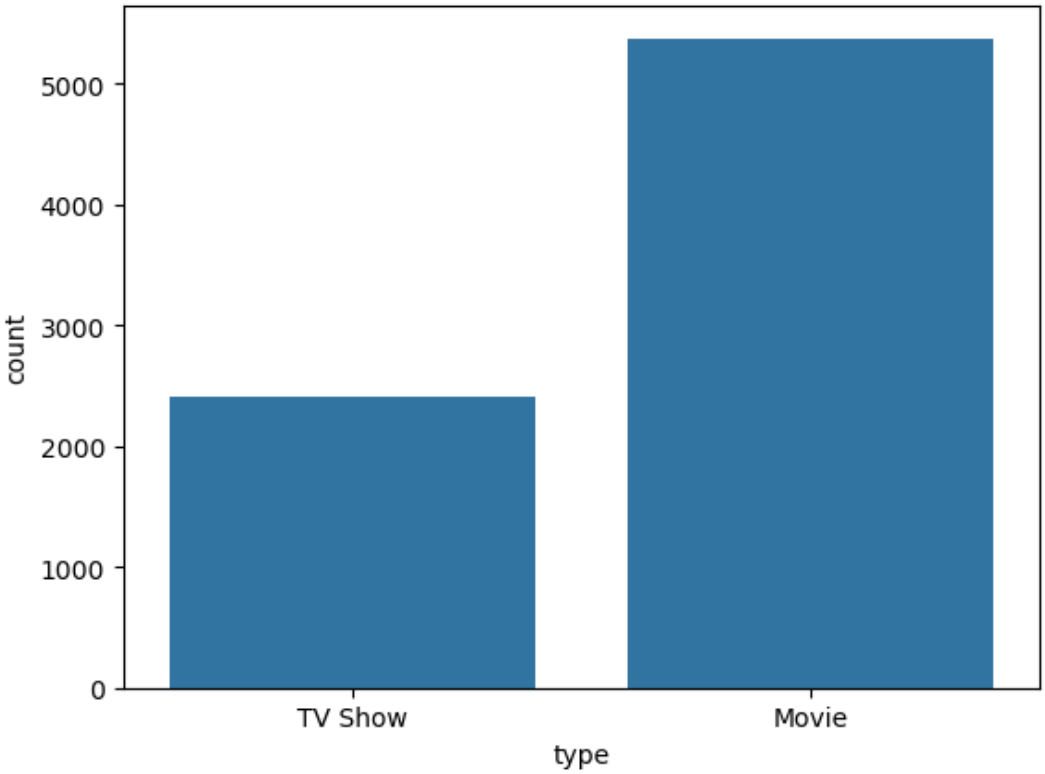
With the data cleaned and enriched, we now move to the visualization phase. This help us identify:

- **The distribution of content types (Movies vs. TV Shows).**
- **Trends in content releases over the years.**
- **Relationship between content duration and its category.**

In [314]:

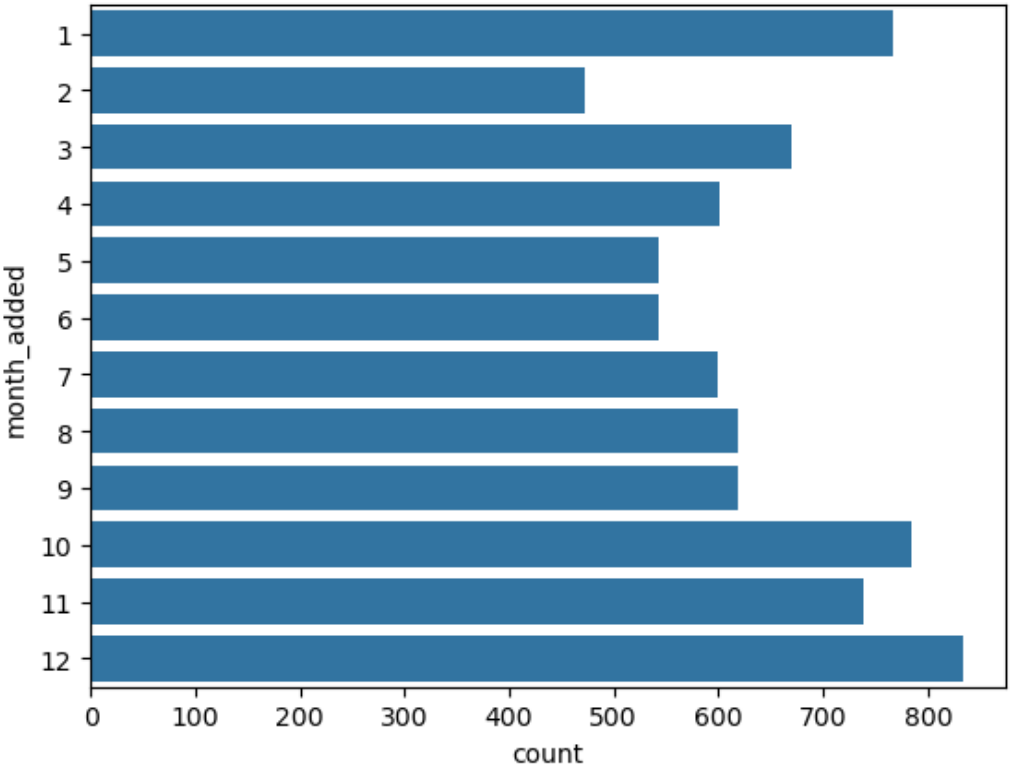
```
sns.countplot(x="type", data=df)
```

```
sns.countplot(x="type",data=df)
plt.show()
```



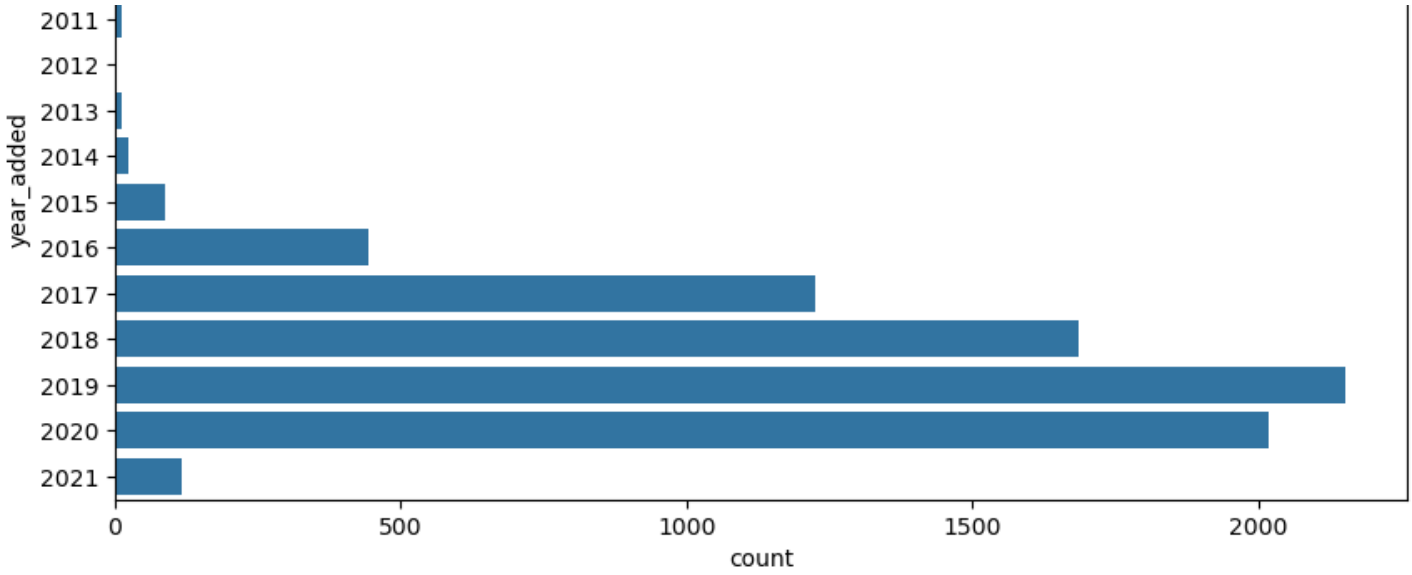
In [315]:

```
sns.countplot(y="month_added",data=df)
plt.show()
```



In [316]:

```
plt.figure(figsize=(10,5))
sns.countplot(y=df["year_added"])
plt.show()
```



In [ ]:

```
```

In [ ]:

```
```

In [317]:

```
df['has_director'] = df['director'].apply(lambda x: 0 if x=='Unknown' else 1)
```

In [318]:

```
df.corr(numeric_only=True)
```

Out[318]:

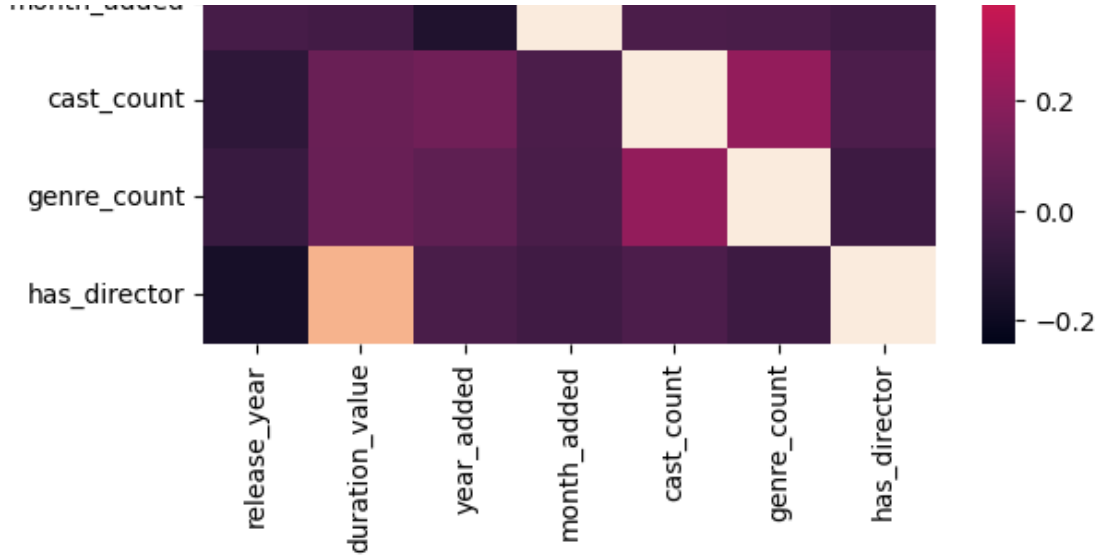
	release_year	duration_value	year_added	month_added	cast_count	genre_count	has_director
release_year	1.000000	-0.240836	0.100115	-0.009661	-0.086585	-0.052931	-0.166603
duration_value	-0.240836	1.000000	0.037102	-0.024913	0.096601	0.092256	0.820335
year_added	0.100115	0.037102	1.000000	-0.135006	0.113470	0.060552	-0.001745
month_added	-0.009661	-0.024913	-0.135006	1.000000	0.001902	-0.002332	-0.028360
cast_count	-0.086585	0.096601	0.113470	0.001902	1.000000	0.219182	0.008707
genre_count	-0.052931	0.092256	0.060552	-0.002332	0.219182	1.000000	-0.041000
has_director	-0.166603	0.820335	-0.001745	-0.028360	0.008707	-0.041000	1.000000

In [319]:

```
sns.heatmap(df.corr(numeric_only=True))
```

Out[319]:

```
<Axes: >
```



Q1] CONTENT-TYPE LABELER:

In [320]:

```
df.head()
```

Out[320]:

	type	title	director	cast	country	release_year	duration_value	genres	description	year_added
0	TV Show	3%	Unknown	João Miguel, Bianca Comparato, Michel Gomes, R...	Brazil	2020	4	International TV Shows, TV Dramas, TV Sci-Fi &...	In a future where the elite inhabit an island ...	2020
1	Movie	1920	Vikram Bhatt	Rajneesh Duggal, Adah Sharma, Indraneil Sengup...	India	2008	143	Horror Movies, International Movies, Thrillers	An architect and his wife move into a castle t...	2017
2	Movie	3 Heroines	Iman Brotoseno	Reza Rahadian, Bunga Citra Lestari, Tara Basro...	Indonesia	2016	124	Dramas, International Movies, Sports Movies	Three Indonesian women break records by becomi...	2019
3	Movie	Blue Mountain State: The Rise of Thadland	Lev L. Spiro	Alan Ritchson, Darin Brooks, James Cade, Rob R...	United States	2016	90	Comedies	New NFL star Thad buys his old teammates' belo...	2016
4	TV Show	Blue Planet II	Unknown	David Attenborough	United Kingdom	2017	1	British TV Shows, Docuseries, Science & Nature TV	This sequel to the award-winning nature series...	2018

In [ ]:

In [ ]:

LOGISTIC - REGRESSION:

In [321]:

```
# Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import pickle


num_features = ['cast_count', 'genre_count', 'duration_value']

cat_features = ['country_main', 'rating_category']

X = df[num_features + cat_features]
y = df['type']

# Split the data into train and test sets (80-20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42
, stratify=y)


num_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])


cat_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore'))
])


preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_transformer, num_features),
        ('cat', cat_transformer, cat_features)
    ])



prediction_pipeline_1 = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(random_state=42, max_iter=1000))
])


prediction_pipeline_1.fit(X_train, y_train)


y_pred = prediction_pipeline_1.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Movie', 'TV Show']))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Save the pipeline model (similar to laptop project)
with open('netflix_type_predictor_pipeline_1.pkl', 'wb') as f:
    pickle.dump(prediction_pipeline_1, f)


# Example prediction function (like app.py in laptop project)
def predict_content_type(cast_count, genre_count, duration_value,
```

```
                            country_main, rating_category):
    # Create input DataFrame
    query_df = pd.DataFrame({
        'cast_count': [cast_count],
        'genre_count': [genre_count],
        'duration_value': [duration_value],
        'country_main': [country_main],
        'rating_category': [rating_category]
    })

    # Load the pipeline
    with open('netflix_type_predictor_pipeline_1.pkl', 'rb') as f:
        pipe = pickle.load(f)

    # Predict
    prediction = pipe.predict(query_df)[0]
    content_type = 'TV Show' if prediction == 1 else 'Movie'
    probability = pipe.predict_proba(query_df)[0]
    conf_movie = probability[0] * 100
    conf_tv = probability[1] * 100

    return f"Predicted Type: {content_type} (Confidence: {max(conf_movie, conf_tv):.2f}%)
"

# Example usage
print("\nExample Prediction:")
result = predict_content_type(cast_count=8, genre_count=3, duration_value=143,
    country_main='India', rating_category='Adult'
)
print(result)
```

```
Accuracy: 0.9961

Classification Report:
              precision    recall  f1-score   support

       Movie       1.00      0.99      1.00      1076
     TV Show       0.99      1.00      0.99       482

    accuracy                           1.00      1558
   macro avg       0.99      1.00      1.00      1558
weighted avg       1.00      1.00      1.00      1558


Confusion Matrix:
[[1070    6]
 [   0  482]]

Example Prediction:
Predicted Type: Movie (Confidence: 100.00%)
```

In [ ]:

NAIVE - BAYES (Bernoulli) :

In [322]:

```
from sklearn.naive_bayes import BernoulliNB

prediction_pipeline_2 = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', BernoulliNB(
    alpha=1.0,
    binarize=0.0,
    fit_prior=True,
    class_prior=None
))
])
```

```
prediction_pipeline_2.fit(X_train, y_train)


y_pred = prediction_pipeline_2.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Movie', 'TV Show']))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Save the pipeline model (similar to laptop project)
with open('netflix_type_predictor_pipeline_2.pkl', 'wb') as f:
    pickle.dump(prediction_pipeline_2, f)
```

```
Accuracy: 0.8954

Classification Report:
              precision    recall  f1-score   support

       Movie       0.98      0.86      0.92      1076
     TV Show       0.76      0.96      0.85       482

    accuracy                           0.90      1558
   macro avg       0.87      0.91      0.89      1558
weighted avg       0.91      0.90      0.90      1558


Confusion Matrix:
[[930 146]
 [ 17 465]]
```

In [ ]:

```

```

KNN:

In [323]:

```
from sklearn.neighbors import KNeighborsClassifier

prediction_pipeline_3 = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', KNeighborsClassifier(
        n_neighbors=5,
        weights='uniform',
        algorithm='auto',
        metric='minkowski',
        p=2
    ))
])

prediction_pipeline_3.fit(X_train, y_train)

y_pred = prediction_pipeline_3.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Movie', 'TV Show']))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
# Save the pipeline model
with open('netflix_type_predictor_knn_pipeline_3.pkl', 'wb') as f:
    pickle.dump(prediction_pipeline_3, f)
```

```
Accuracy: 0.9891

Classification Report:
              precision    recall  f1-score   support

       Movie       1.00      0.99      0.99      1076
     TV Show       0.97      1.00      0.98       482

    accuracy                           0.99      1558
   macro avg       0.98      0.99      0.99      1558
weighted avg       0.99      0.99      0.99      1558


Confusion Matrix:
[[1060   16]
 [   1  481]]
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

CONCLUSION: Highest accuracy is acheived through Logistic Regression.

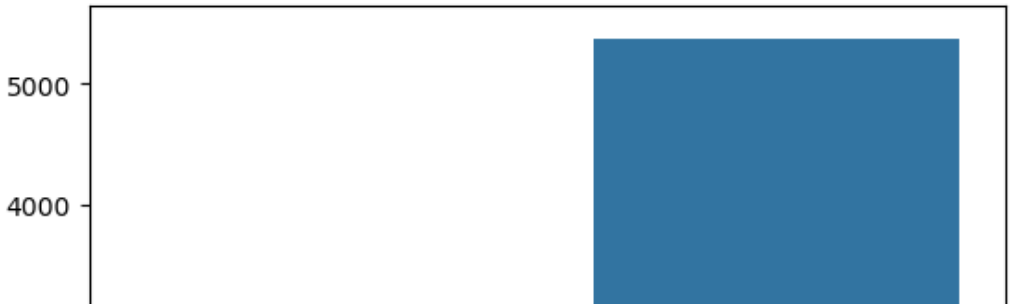
In [ ]:

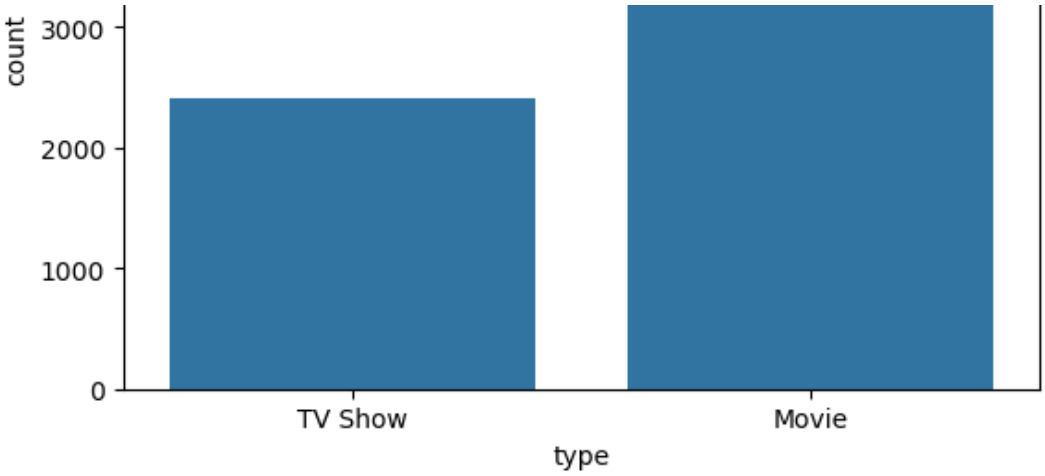
In [ ]:

Q2] Predict whether a piece of content is a Movie or a TV Show based on features like cast-size, country, and release trends?

In [324]:

```
sns.countplot(x=df["type"])
plt.show()
```





In [325]:

```
data = df.groupby("type")["genre_count"].sum().reset_index()
data
```

Out[325]:

	type	genre_count
0	Movie	11546
1	TV Show	5525

In [326]:

```
data = df.groupby("type")["cast_count"].sum().reset_index()
data
```

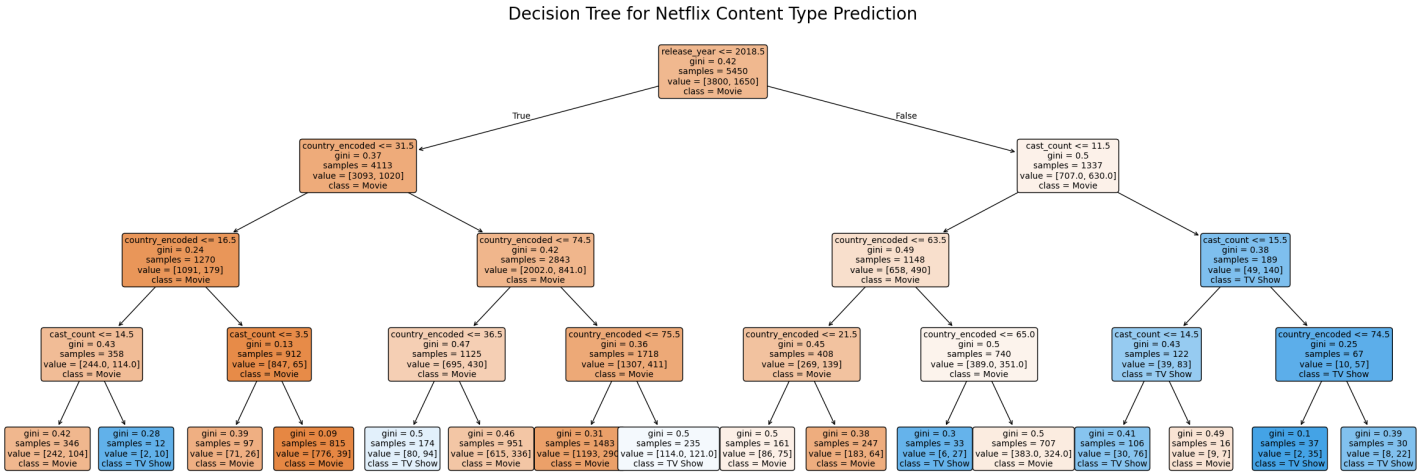
Out[326]:

	type	cast_count
0	Movie	38738
1	TV Show	17935

In [327]:

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# 1. Preprocessing categorical features
# We extract the first country and first genre to simplify the categories
df['first_country'] = df['country'].fillna('Unknown').apply (lambda x: x.split(',')[0])

# 2. Encode strings to numbers
le_country = LabelEncoder()

df['country_encoded'] = le_country.fit_transform(df['first_country'])



feature_cols = ["cast_count", "country_encoded", "release_year"]
X = df[feature_cols]
y = df["type"]


X_train , X_test , y_train, y_test = train_test_split(X,y, test_size=0.3,random_state=42
)


model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train , y_train)
```

```
plt.figure(figsize=(30,10))
plot_tree(model,
          feature_names=feature_cols,
          class_names=["Movie", "TV Show"],
          filled=True,
          rounded=True,
          fontsize=10,
          precision=2)

plt.title("Decision Tree for Netflix Content Type Prediction", fontsize=20)
plt.show()


print("Accuracy:",model.score(X_test, y_test))
```



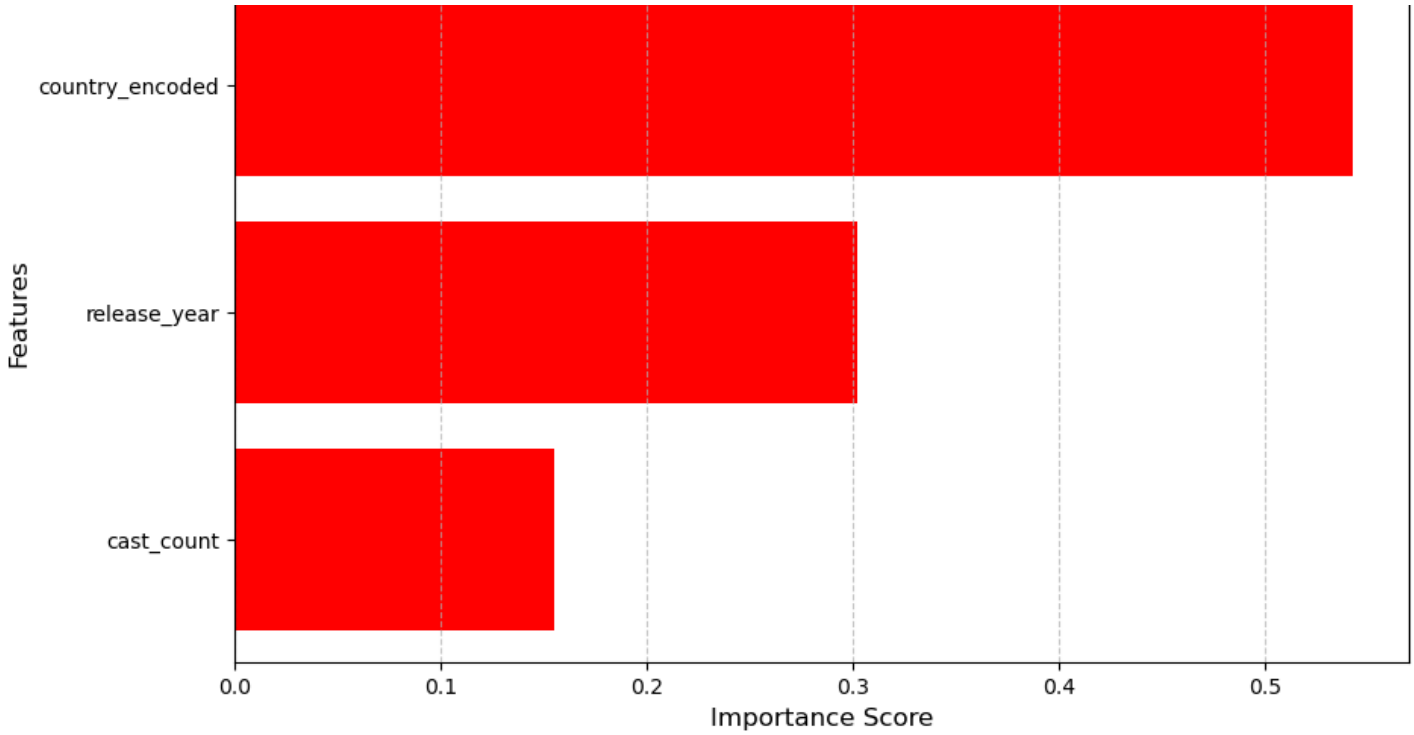

```
Accuracy: 0.7064612751390672
```

In [ ]:

```

```

In [328]:

```
# 1. Create a DataFrame to hold the feature names and their corresponding importance scor
es
feature_importance_df = pd.DataFrame({
    'Feature': feature_cols,
    'Importance': model.feature_importances_
}).sort_values(by='Importance', ascending=True)

# 2. Plot the Feature Importance
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df['Feature'], feature_importance_df['Importance'], color='re
d')

plt.title('Feature Importance for Netflix Content Type Prediction', fontsize=16)
plt.xlabel('Importance Score', fontsize=12)
plt.ylabel('Features', fontsize=12)
plt.grid(axis='x', linestyle='--', alpha=0.7)

plt.show()

# 3. Print the values for reference
# Create the sorted dataframe and drop the old index
importance_display = feature_importance_df.sort_values(by='Importance', ascending=False)
.reset_index(drop=True)

# Display the clean table
importance_display.reset_index()
```



Out[328]:

	index	Feature	Importance
0	**0**	**country_encoded**	**0.542345**
1	**1**	**release_year**	**0.302328**
2	**2**	**cast_count**	**0.155326**

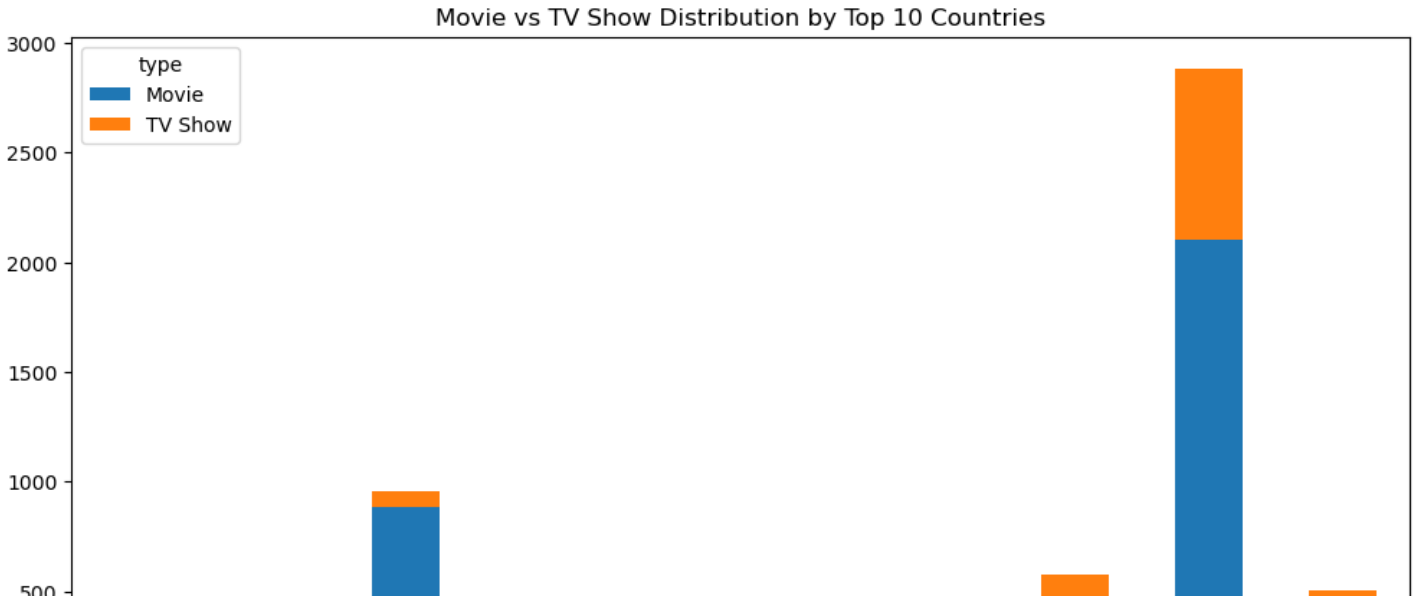
In [ ]:

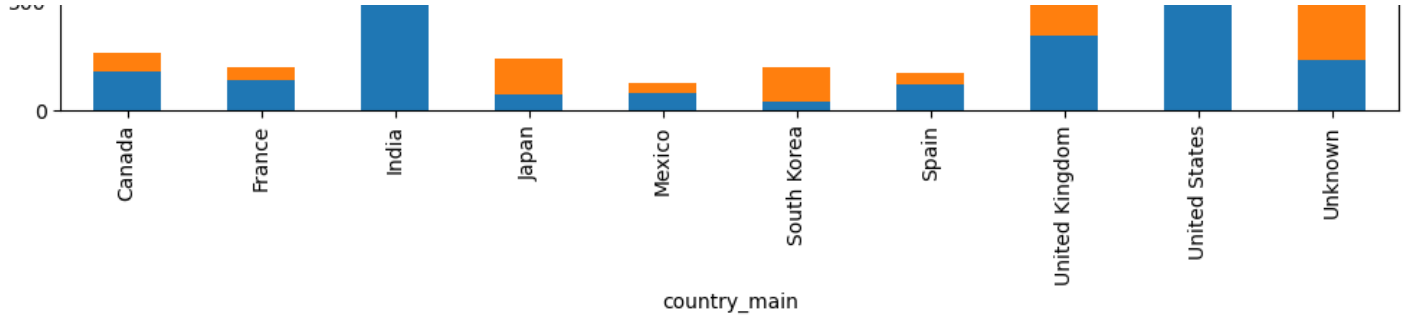
```

```

Q3] Which countries dominate Netflix content, and how does the Movie vs TV-Show ratio change across the top 10 countries?

In [329]:

```
top10_countries = df['country_main'].value_counts().head(10).index
country_type = df[df['country_main'].isin(top10_countries)].groupby(['country_main', 'ty
pe']).size().unstack()
country_type.plot(kind='bar', stacked=True, figsize=(12,6))
plt.title('Movie vs TV Show Distribution by Top 10 Countries')
plt.show()
```

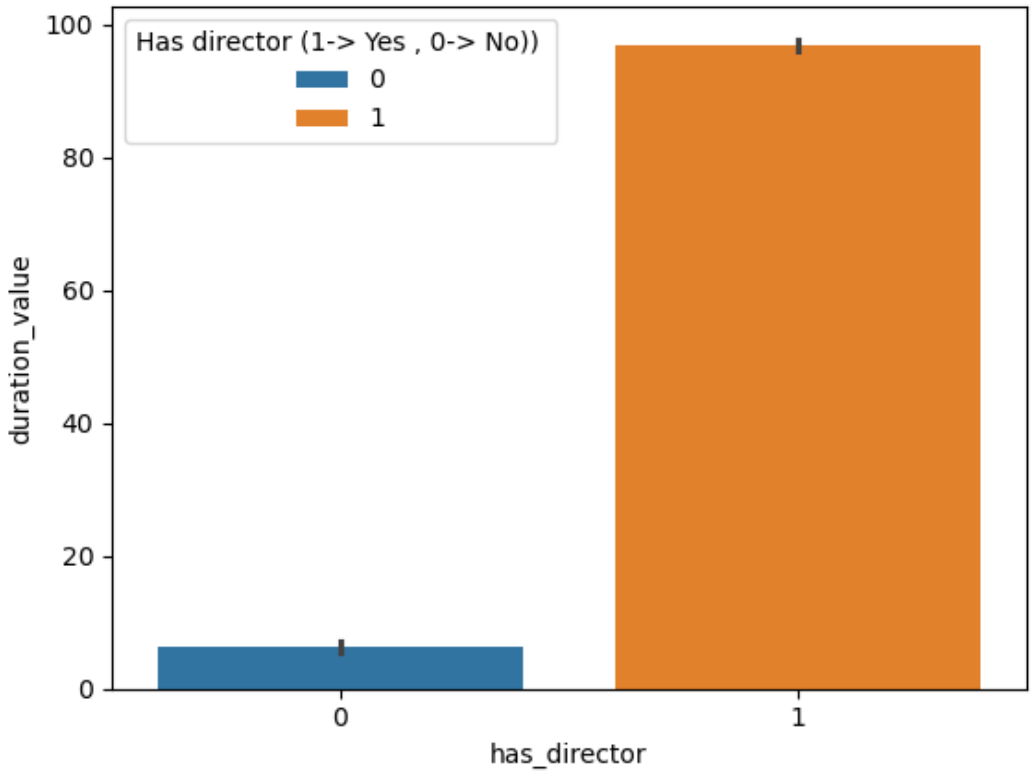




Q4] Do titles with a known director have significantly higher cast_count, genre_count, or longer duration than “Unknown” director titles?

In [330]:

```
df.groupby('has_director')[['cast_count', 'genre_count', 'duration_value']].mean().round
(2)
sns.barplot(x='has_director', y='duration_value',hue="has_director", data=df)
plt.legend(title="Has director (1-> Yes , 0-> No)) ")
plt.show()
```



In [331]:

```
df.head()
```

Out[331]:

	type	title	director	cast	country	release_year	duration_value	genres	description	year_added
0	TV Show	3%	Unknown	João Miguel, Bianca Comparato, Michel Gomes, R...	Brazil	2020	4	International TV Shows, TV Dramas, TV Sci-Fi &...	In a future where the elite inhabit an island ...	2020
1	Movie	1920	Vikram Bhatt	Rajneesh Duggal, Adah Sharma, Indraneil Sengup...	India	2008	143	Horror Movies, International Movies, Thrillers	An architect and his wife move into a castle t...	2017
				Reza				Dramas	Three	

	type	title	director	cast	country	release_year	duration_value	genres	description	year_added
2	Movie	3 Heroines	Iman Brotoseno	Reza Rahadian, Bunga Citra Lestari, Tara Basro...	Indonesia	2016	124	Dramas, International Movies, Sports Movies	Indonesian women break records by becomi...	2019
3	Movie	Blue Mountain State: The Rise of Thadland	Lev L. Spiro	Alan Ritchson, Darin Brooks, James Cade, Rob R...	United States	2016	90	Comedies	New NFL star Thad buys his old teammates' belo...	2016
4	TV Show	Blue Planet II	Unknown	David Attenborough	United Kingdom	2017	1	British TV Shows, Docuseries, Science & Nature TV	This sequel to the award-winning nature series...	2018

CONTENT- RECOMMENDATION SYSTEM:

In [332]:

```
import pandas as pd
import pickle
import numpy as np

from nltk.tokenize import word_tokenize
from gensim.models import Word2Vec
from sklearn.metrics.pairwise import cosine_similarity

# ----------------------------
# STEP 1: Create tags (same as before)
# ----------------------------
df['tags'] = (
    df['title'].fillna('') + " " +
    df['genres'].fillna('') + " " +
    df['description'].fillna('') + " " +
    df['cast'].fillna('') + " " +
    df['director'].fillna('') + " " +
    df['country_main'].fillna('') + " " +
    df['rating_category'].fillna('') + " " +
    df['type'].fillna('')
)

df['tags'] = df['tags'].str.lower()

# ----------------------------
# STEP 2: Tokenization
# ----------------------------
df['tokens'] = df['tags'].apply(lambda x: word_tokenize(x))

# ----------------------------
# STEP 3: Train Word2Vec Model
# ----------------------------
w2v_model = Word2Vec(
    sentences=df['tokens'],
    vector_size=100,
    window=5,
    min_count=1,
    workers=4
)

# ----------------------------
# STEP 4: Convert each content into vector
# ----------------------------
def get_doc_vector(tokens):
    vectors = []
    for word in tokens:
        if word in w2v_model.wv:
```

```python
            vectors.append(w2v_model.wv[word])

    if len(vectors) == 0:
        return np.zeros(100)

    return np.mean(vectors, axis=0)

df['vector'] = df['tokens'].apply(get_doc_vector)

# Convert to matrix
vectors = np.vstack(df['vector'].values)

# ----------------------------
# STEP 5: Recommendation Class
# ----------------------------
class RecommendationModel:
    def __init__(self, vectors, df, w2v_model):
        self.vectors = vectors
        self.df = df.reset_index(drop=True)
        self.w2v_model = w2v_model

    def text_to_vector(self, text):
        tokens = word_tokenize(text.lower())
        return get_doc_vector(tokens)

    def recommend(self, content, top_n=10):
        mask = self.df['title'].str.contains(content, case=False, na=False)

        if mask.any():
            idx = self.df[mask].index[0]
            query_vec = self.vectors[idx].reshape(1, -1)
            print(" Found exact title → using full features")
        else:
            query_vec = self.text_to_vector(content).reshape(1, -1)

        similarity = cosine_similarity(query_vec, self.vectors)[0]

        content_list = sorted(
            list(enumerate(similarity)),
            reverse=True,
            key=lambda x: x[1]
        )[1:top_n+1]

        print(f"\n\nRecommendations similar to: **{content}**\n")

        for rank, (i, score) in enumerate(content_list, 1):
            row = self.df.iloc[i]
            print(f"{rank:2}) • {row['title']}  "
                  f"| {row['type']}  "
                  f"| {row['genres'][:70]}...")

# ----------------------------
# STEP 6: Save & Load Model
# ----------------------------
model = RecommendationModel(vectors, df, w2v_model)

with open("content_recommendation_word2vec.pkl", "wb") as f:
    pickle.dump(model, f)

with open("content_recommendation_word2vec.pkl", "rb") as f:
    loaded_model = pickle.load(f)

# ----------------------------
# STEP 7: Test
# ----------------------------
loaded_model.recommend("Kesari 2")
loaded_model.recommend("Dune")
loaded_model.recommend("Stranger Things")
loaded_model.recommend("Article 15")
loaded_model.recommend("Raone")
loaded_model.recommend("3 Idiots")
loaded_model.recommend("Dhurandhar")
```

```
loaded_model.recommend("Dhurandhar - The Revenge")
```

```
Recommendations similar to: **Kesari 2**

 1) • Untamed Romania  | Movie  | Children & Family Movies, Documentaries, International
Movies...
 2) • Magnus  | Movie  | Children & Family Movies, Documentaries, International Movies...
 3) • Luccas Neto in: Children's Day  | Movie  | Children & Family Movies, Comedies, Musi
c & Musicals...
 4) • This Was Tomorrow  | Movie  | Documentaries, International Movies, Music & Musicals
...
 5) • Covered: Alive in Asia  | Movie  | Documentaries, Faith & Spirituality, Music & Mus
icals...
 6) • Solo: A Star Wars Story (Spanish Version)  | Movie  | Action & Adventure, Children
& Family Movies, Sci-Fi & Fantasy...
 7) • Vettai  | Movie  | Action & Adventure, International Movies, Music & Musicals...
 8) • Solo: A Star Wars Story  | Movie  | Action & Adventure, Children & Family Movies, S
ci-Fi & Fantasy...
 9) • Mission Istaanbul: Darr Ke Aagey Jeet Hai  | Movie  | Action & Adventure, Internati
onal Movies, Music & Musicals...
10) • Baahubali: The Beginning (Hindi Version)  | Movie  | Action & Adventure, Dramas, In
ternational Movies...


Recommendations similar to: **Dune**

 1) • 1920  | Movie  | Horror Movies, International Movies, Thrillers...
 2) • 3 Heroines  | Movie  | Dramas, International Movies, Sports Movies...
 3) • Blue Mountain State: The Rise of Thadland  | Movie  | Comedies...
 4) • Blue Planet II  | TV Show  | British TV Shows, Docuseries, Science & Nature TV...
 5) • Blue Ruin  | Movie  | Independent Movies, Thrillers...
 6) • Blue Streak  | Movie  | Action & Adventure, Comedies...
 7) • Blue Valentine  | Movie  | Dramas, Independent Movies, Romantic Movies...
 8) • BluffMaster!  | Movie  | Comedies, International Movies, Romantic Movies...
 9) • Blurred Lines: Inside the Art World  | Movie  | Documentaries...
10) • BNA  | TV Show  | Anime Series, International TV Shows...
 Found exact title → using full features


Recommendations similar to: **Stranger Things**

 1) • Dueños del paraíso  | TV Show  | Crime TV Shows, International TV Shows, Spanish-La
nguage TV Shows...
 2) • Control Z  | TV Show  | International TV Shows, Spanish-Language TV Shows, TV Drama
s...
 3) • Club of Crows  | TV Show  | International TV Shows, Spanish-Language TV Shows, TV C
omedies...
 4) • Unauthorized Living  | TV Show  | Crime TV Shows, International TV Shows, Spanish-L
anguage TV Shows...
 5) • Lost & Found Music Studios  | TV Show  | Kids' TV, TV Comedies, TV Dramas...
 6) • Between  | TV Show  | International TV Shows, TV Dramas, TV Mysteries...
 7) • Twin Peaks  | TV Show  | Classic & Cult TV, Crime TV Shows, TV Dramas...
 8) • Arrow  | TV Show  | Crime TV Shows, TV Action & Adventure...
 9) • InuYasha  | TV Show  | Anime Series, International TV Shows...
10) • Vampire Knight  | TV Show  | Anime Series, International TV Shows, Romantic TV Show
s...
 Found exact title → using full features


Recommendations similar to: **Article 15**

 1) • Chupan Chupai  | Movie  | Comedies, International Movies...
 2) • Apaharan  | Movie  | Dramas, International Movies...
 3) • Made in China  | Movie  | Comedies, Dramas, International Movies...
 4) • Delhi 6  | Movie  | Dramas, International Movies...
 5) • Saheb Biwi Aur Gangster Returns  | Movie  | Dramas, International Movies, Romantic
Movies...
 6) • Chance Pe Dance  | Movie  | Comedies, Dramas, International Movies...
 7) • Lock  | Movie  | Dramas, International Movies...
 8) • An Upper Egyptian  | Movie  | Comedies, Dramas, International Movies...
 9) • Judwaa  | Movie  | Comedies, International Movies, Romantic Movies...
```

```
10) • Bhoot  | Movie  | Horror Movies, International Movies, Thrillers...


Recommendations similar to: **Raone**

 1) • 1920  | Movie  | Horror Movies, International Movies, Thrillers...
 2) • 3 Heroines  | Movie  | Dramas, International Movies, Sports Movies...
 3) • Blue Mountain State: The Rise of Thadland  | Movie  | Comedies...
 4) • Blue Planet II  | TV Show  | British TV Shows, Docuseries, Science & Nature TV...
 5) • Blue Ruin  | Movie  | Independent Movies, Thrillers...
 6) • Blue Streak  | Movie  | Action & Adventure, Comedies...
 7) • Blue Valentine  | Movie  | Dramas, Independent Movies, Romantic Movies...
 8) • BluffMaster!  | Movie  | Comedies, International Movies, Romantic Movies...
 9) • Blurred Lines: Inside the Art World  | Movie  | Documentaries...
10) • BNA  | TV Show  | Anime Series, International TV Shows...
 Found exact title → using full features


Recommendations similar to: **3 Idiots**

 1) • Baazaar  | Movie  | Dramas, International Movies...
 2) • Peepli Live  | Movie  | Comedies, Dramas, Independent Movies...
 3) • Aarakshan  | Movie  | Dramas, International Movies...
 4) • Yuva  | Movie  | Dramas, International Movies...
 5) • Yeh Ballet  | Movie  | Dramas, International Movies...
 6) • Singapore  | Movie  | Comedies, Dramas, International Movies...
 7) • Mohamed Hussein  | Movie  | Comedies, International Movies...
 8) • Son Of Adam  | Movie  | Dramas, International Movies, Thrillers...
 9) • Bypass Road  | Movie  | International Movies, Thrillers...
10) • Lucky: No Time for Love  | Movie  | Dramas, International Movies, Romantic Movies..
.


Recommendations similar to: **Dhurandhar**

 1) • 1920  | Movie  | Horror Movies, International Movies, Thrillers...
 2) • 3 Heroines  | Movie  | Dramas, International Movies, Sports Movies...
 3) • Blue Mountain State: The Rise of Thadland  | Movie  | Comedies...
 4) • Blue Planet II  | TV Show  | British TV Shows, Docuseries, Science & Nature TV...
 5) • Blue Ruin  | Movie  | Independent Movies, Thrillers...
 6) • Blue Streak  | Movie  | Action & Adventure, Comedies...
 7) • Blue Valentine  | Movie  | Dramas, Independent Movies, Romantic Movies...
 8) • BluffMaster!  | Movie  | Comedies, International Movies, Romantic Movies...
 9) • Blurred Lines: Inside the Art World  | Movie  | Documentaries...
10) • BNA  | TV Show  | Anime Series, International TV Shows...


Recommendations similar to: **Dhurandhar - The Revenge**

 1) • Making Unorthodox  | Movie  | Documentaries, International Movies...
 2) • Little Baby Bum: Go Buster  | Movie  | Children & Family Movies...
 3) • Harlem Globetrotters: The Team That Changed the World  | Movie  | Documentaries, Sp
orts Movies...
 4) • The Last Man on the Moon  | Movie  | Documentaries...
 5) • Bobby Kennedy for President  | TV Show  | Docuseries...
 6) • Perfect Bid: The Contestant Who Knew Too Much  | Movie  | Documentaries...
 7) • Calico Critters: A Town of Dreams  | Movie  | Children & Family Movies...
 8) • Bugs  | Movie  | Documentaries, International Movies...
 9) • Merata: How Mum Decolonised the Screen  | Movie  | Documentaries...
10) • Jimmy Carr: The Best of Ultimate Gold Greatest Hits  | Movie  | Stand-Up Comedy...
```

In [ ]:

Q5] Can we forecast how many titles Netflix will add in 2022–2025?

In [333]:

```
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error

yearly_data = df.groupby('year_added').size().reset_index(name='title_count')
yearly_data = yearly_data[yearly_data['year_added'] <= 2021]

X = yearly_data[['year_added']]
y = yearly_data['title_count']

# □ DO THIS ONLY ONCE
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

future_years = np.array([[2022], [2023], [2024], [2025]])
```

a) Linear Regression:

In [334]:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression

linear_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('poly', PolynomialFeatures(degree=1)),
    ('model', LinearRegression())
])

linear_pipeline.fit(X_train, y_train)

y_pred_linear = linear_pipeline.predict(X_test)
linear_forecast = linear_pipeline.predict(future_years)
```

b) Adaboost:

In [335]:

```
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeRegressor

ada_pipeline = Pipeline([
    ('model', AdaBoostRegressor(
        estimator=DecisionTreeRegressor(max_depth=3),
        n_estimators=100,
        learning_rate=0.5,
        random_state=42
    ))
])

ada_pipeline.fit(X_train, y_train)

y_pred_ada = ada_pipeline.predict(X_test)
ada_forecast = ada_pipeline.predict(future_years)
```

c) Decision tree:

In [336]:

```
from sklearn.tree import DecisionTreeRegressor

tree_pipeline = Pipeline([
    ('model', DecisionTreeRegressor(
```

```
        max_depth=3,
        random_state=42
    ))
])

tree_pipeline.fit(X_train, y_train)

y_pred_tree = tree_pipeline.predict(X_test)
tree_forecast = tree_pipeline.predict(future_years)
```

In [337]:

```
forecast_df = pd.DataFrame({
    'Year': [2022, 2023, 2024, 2025],
    'Linear': linear_forecast.astype(int),
    'AdaBoost': ada_forecast.astype(int),
    'Decision Tree': tree_forecast.astype(int)
})

print(forecast_df)


print("\n\n\n")

results = pd.DataFrame({
    'Model': ['Linear', 'AdaBoost', 'Decision Tree'],
    'R2 Score': [
        r2_score(y_test, y_pred_linear),
        r2_score(y_test, y_pred_ada),
        r2_score(y_test, y_pred_tree)
    ],
    'MAE': [
        mean_absolute_error(y_test, y_pred_linear),
        mean_absolute_error(y_test, y_pred_ada),
        mean_absolute_error(y_test, y_pred_tree)
    ]
})

print(results)
```

```
   Year  Linear  AdaBoost  Decision Tree
0  2022    1283       117            117
1  2023    1400       117            117
2  2024    1517       117            117
3  2025    1634       117            117




           Model  R2 Score         MAE
0         Linear  0.186415  701.208477
1       AdaBoost  0.643207  418.583333
2  Decision Tree  0.643199  419.055556
```

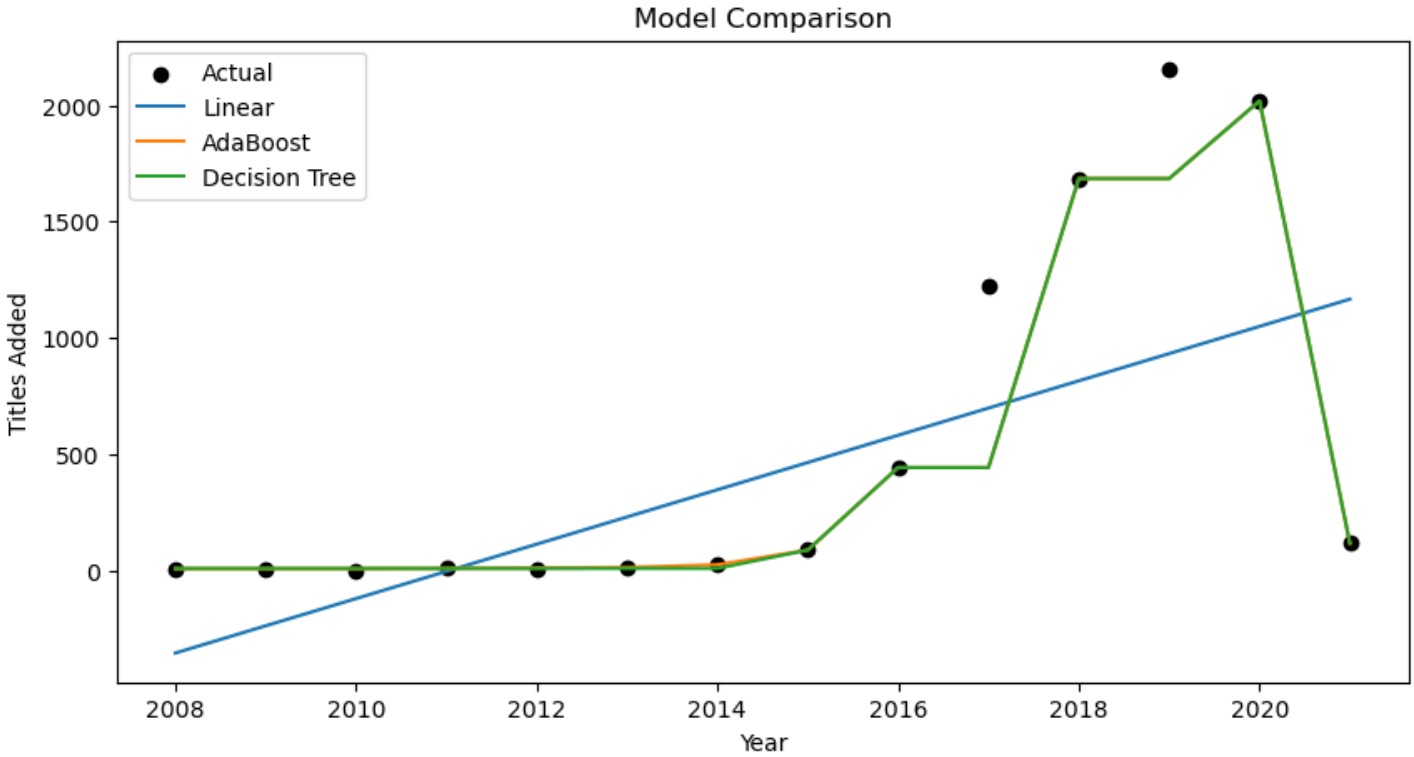
CONCLUSION] Tree-based models such as DecisionTreeRegressor and AdaBoostRegressor were tested but produced similar and unrealistic forecasts. This is because tree-based models do not extrapolate beyond the training data and tend to create step-like predictions, making them unsuitable for time-series forecasting tasks with limited features.

In [ ]:

In [338]:

```
# Sort for smooth plotting
sorted_idx = X['year_added'].argsort()
X_sorted = X.iloc[sorted_idx]
```

```
plt.figure(figsize=(10,5))

plt.scatter(X, y, color='black', label='Actual')

plt.plot(X_sorted, linear_pipeline.predict(X_sorted), label='Linear')
plt.plot(X_sorted, ada_pipeline.predict(X_sorted), label='AdaBoost')
plt.plot(X_sorted, tree_pipeline.predict(X_sorted), label='Decision Tree')

plt.legend()
plt.title('Model Comparison')
plt.xlabel('Year')
plt.ylabel('Titles Added')

plt.show()
```



In [ ]:

CONCLUSION] Linear regression performed best for trend modeling, while tree-based models failed to extrapolate future values due to their step-wise nature.

In [ ]:

In [ ]:

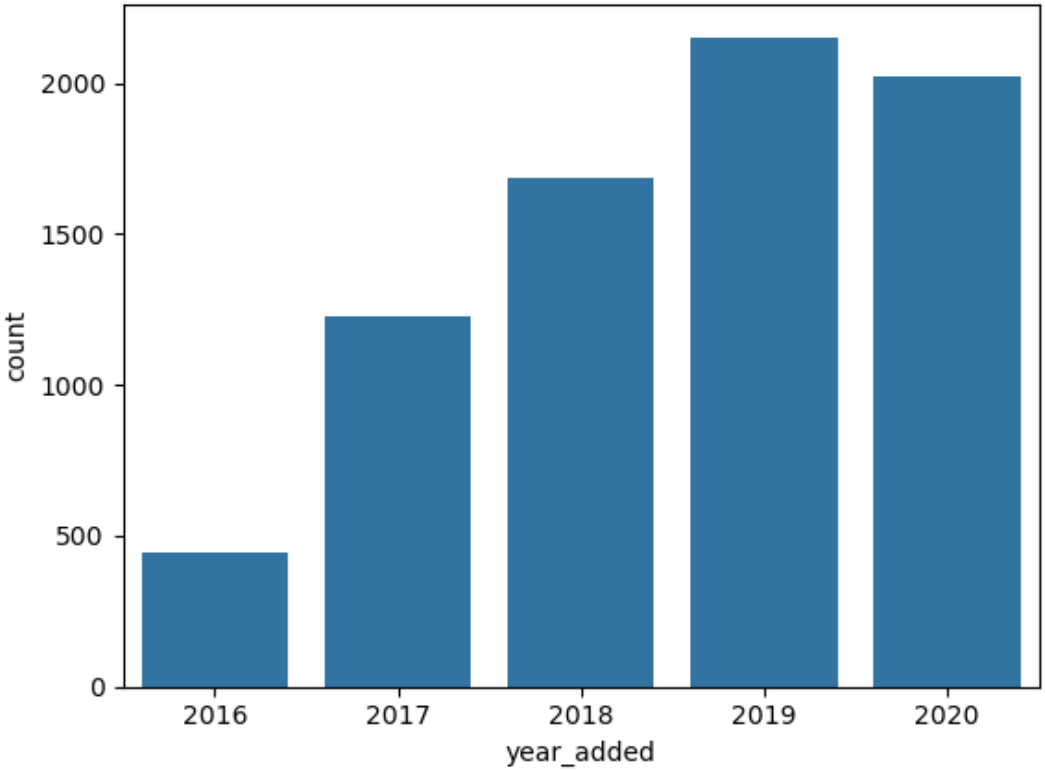
In [ ]:

In [ ]:

Q6] How is Netflix content growing over time?

In [339]:

```
data = df["year_added"].value_counts().reset_index()
data = data.head(5)
sns.barplot(x=data["year_added"],y=data["count"])
plt.show()
```



In [ ]:

```

```

In [340]:

```
yearly_data.sort_values(by="title_count",ascending=False).head(5)
```

Out[340]:

	year_added	title_count
11	2019	2153
12	2020	2019
10	2018	1685
9	2017	1225
8	2016	443

CONCLUSION] Netflix content has shown a consistent upward trend over the years, with significant growth in recent years. Linear models were able to capture this trend effectively, while tree-based models failed due to their inability to extrapolate beyond training data. This highlights the importance of selecting models based on problem type rather than complexity.

In [ ]:

```

```

In [ ]:

```

```

Q7] Who are the top contributing directors?

In [341]:

```
data = df["director"].value_counts().reset_index().head()
data = data[data["director"]!="Unknown"]
data
```

Out[341]:

	director	count
1	Raúl Campos, Jan Suter	18
2	Marcus Raboy	16
3	Jay Karas	14
4	Cathy Garcia-Molina	13

CONCLUSION] The top contributing directors include:

1) Raúl Campos 2) Jan Suter 3) Marcus Raboy 4) Jay Karas 5) Cathy Garcia-Molina

In [ ]:

```

```

Q8] Which countries contribute most to the content-library?

In [342]:

```
data = df["first_country"].value_counts().reset_index().head()
data = data[data["first_country"]!="Unknown"]
data
```

Out[342]:

	first_country	count
0	United States	2883
1	India	956
2	United Kingdom	577
4	Canada	259

CONCLUSION] The top contributing countries are:

1) United States 2) India 3) United Kingdom 4) Canada

In [ ]:

```

```

In [ ]:

```

```

Q9] Is there a statistically significant difference in the average duration of Netflix content between Movies and TV Shows?

In [343]:

```
from scipy.stats import ttest_ind
```

```
# Split data into two groups
movies = df[df['type'] == 'Movie']['duration_value']
tv_shows = df[df['type'] == 'TV Show']['duration_value']

# Perform Independent T-Test
t_stat, p_value = ttest_ind(movies, tv_shows, equal_var=False)

# Print results
print("T-Statistic:", t_stat)
print("P-Value:", p_value)

# Decision
alpha = 0.05

if p_value < alpha:
    print("\nReject Null Hypothesis (H0)")
    print("There is a statistically significant difference in duration.")
else:
    print("\nFail to Reject Null Hypothesis (H0)")
    print("No significant difference in duration.")
```

```
T-Statistic: 249.79900942857398
P-Value: 0.0

Reject Null Hypothesis (H0)
There is a statistically significant difference in duration.
```

CONCLUSION] There is a significant difference in the durations of tv-shows and movies.

In [ ]:

```

```

Q10] Can Netflix content be grouped into meaningful clusters based on features like genre, cast size, and duration?

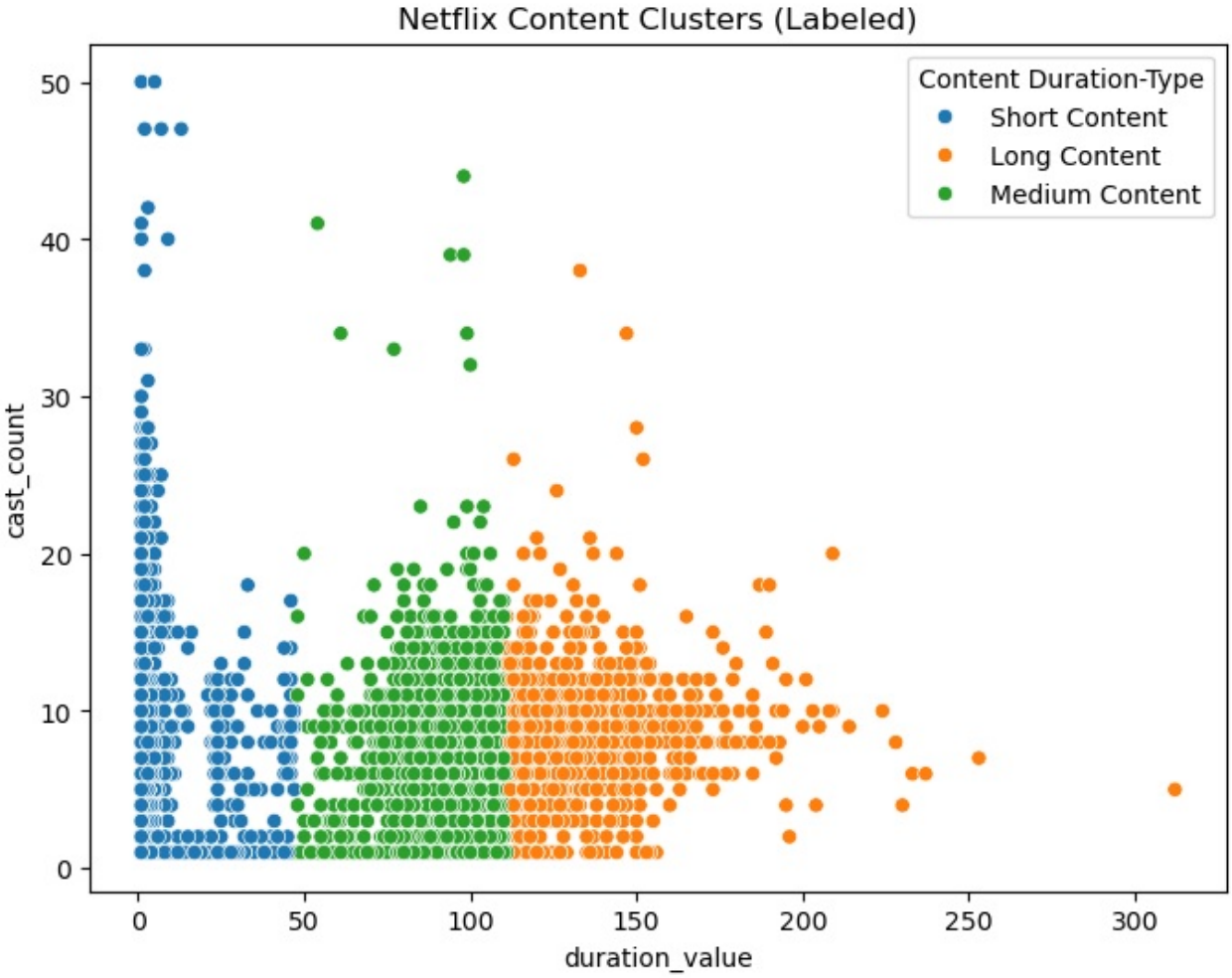
In [344]:

```
from sklearn.cluster import KMeans

X = df[['duration_value', 'cast_count', 'genre_count']]

kmeans = KMeans(n_clusters=3)
df['cluster'] = kmeans.fit_predict(X)

cluster_labels = {
    0: "Short Content",
    1: "Medium Content",
    2: "Long Content"
}

df['cluster_label'] = df['cluster'].map(cluster_labels)
```

In [345]:

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

sns.scatterplot(
    x='duration_value',
    y='cast_count',
    hue='cluster_label',
    data=df
)
```

```
plt.legend(title="Content Duration-Type")
plt.title("Netflix Content Clusters (Labeled)")
plt.show()
```



In [346]:

```
for label in df['cluster_label'].unique():
    subset = df[df['cluster_label'] == label]
    print(f"\n{label}:")
    print("Avg Duration:", subset['duration_value'].mean())
    print("Avg Cast:", subset['cast_count'].mean())
```

```
Short Content:
Avg Duration: 4.443435489973515
Avg Cast: 7.244040862656073

Long Content:
Avg Duration: 132.37158105403603
Avg Cast: 8.66977985323549

Medium Content:
Avg Duration: 90.01097393689986
Avg Cast: 6.730041152263374
```

CONCLUSION] Netflix Content Clustering Analysis

Using K-Means clustering, Netflix content was successfully grouped into three meaningful clusters based on duration, cast size, and genre count. These clusters were interpreted and labeled as Short Content, Medium Content, and Long Content.

The analysis revealed that duration is the most influential feature, as clusters are primarily separated along the duration axis. This indicates that content length plays a major role in defining content categories on the platform.

- **Short Content: Holds the least runtime with 7 casts on average.**
- **Medium Content: Holds the highest runtime with 9 casts (approx.) on average.**

- **Long Content**: Holds moderate runtime with 7 casts (approx.) on average.

Business Insight:

This highlights how content naturally segments into different consumption patterns, allowing the company to:

- **Produce the right mix of short, medium, and long content instead of focusing on just one type.**
- **Tailor recommendations based on content type clusters**
- **Balance investment between short-form and long-form content**

In [ ]:

```

```

In [ ]:

```

```

In [ ]:

```

```